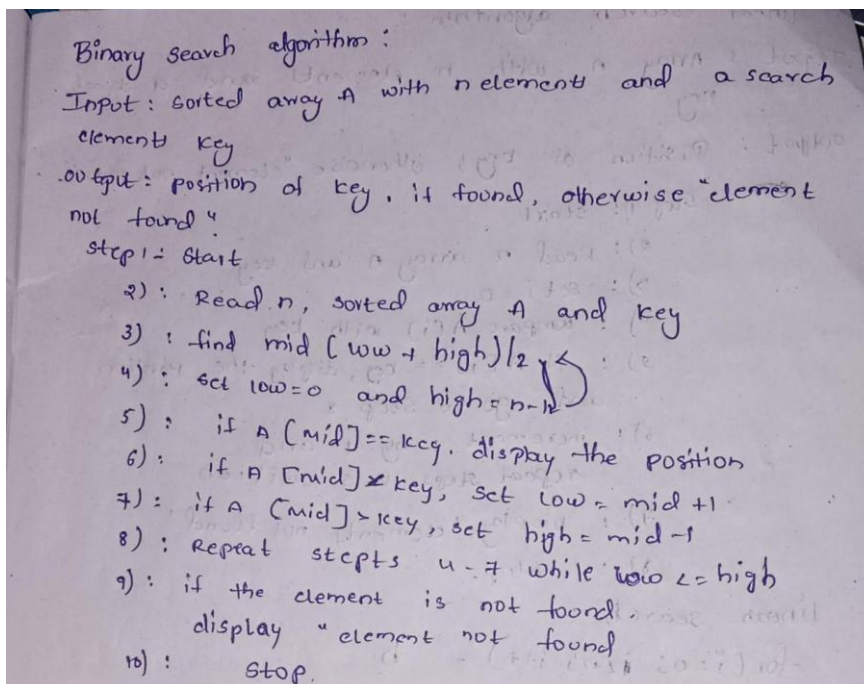


PRACTICAL - 2

AIM :Implementation and Time analysis of linear and binary search algorithm

Binary Search Algorithm :



code :

```
#include <iostream> using  
namespace std;
```

```
int main() { int n,  
arr[100], key;  
int low, high, mid;
```

```
cout << "Enter size of array: ";  
cin >> n;
```

```
cout << "Enter sorted array elements: ";
```

```
for(int i = 0; i < n; i++)
cin >> arr[i];

cout << "Enter element to search: ";
cin >> key;

low = 0;
high = n - 1;

while(low <= high)
{
    mid = (low + high) / 2;

    if(arr[mid] == key)
    {
        cout << "Element found at position " << mid + 1;
        return 0;
    }
    else if(arr[mid] < key)
    {
        low = mid + 1;
    }
    else
    {
        high = mid - 1;
    }
}

cout << "Element not found";

return 0;
}
```

OUTPUT :

```

Enter size of array: 5
Enter sorted array elements: 3
4
6
7
2
Enter element to search: 7
Element found at position 4

...Program finished with exit code 0
Press ENTER to exit console.

```

time complexity : Step Count / Frequency Count Method)
Best case :

Binary search
(Best case)
 $mid = (low + high) / 2$
if $arr[mid] == key$
return mid
 $T(n) = 1$
 $= O(1)$

Average case : Key is found after several divisions

$low = 0$; $high = n - 1$
while ($low \leq high$) {
 $mid = (low + high) / 2$;
 if ($arr[mid] == key$)
 return mid ;
 else if ($arr[mid] < key$)
 $low = mid + 1$;
 else
 $high = mid - 1$;
}
return -1 ;
 $T(n) = O(\log n)$

Worst case : Key is found at the last division / not found

```
low = 0; — 1
high = n-1; — 1
while (low <= high) { —  $O(\log n)$ 
    mid = (low+high)/2; — 1
    if (arr[mid] == key) — 1
        return mid; }
    else if (arr[mid] < key) — 1
        low = mid+1; }
    else {
        high = mid-1; — 1
    }
}
return -1; — 1
T(n) =  $O(\log n)$ 
```

Linear Search Algorithm :

Linear search algorithm :
Input : Array A with n elements and a search key
Output : position of key; otherwise "element not found".

- Step 1): start
- 2): Read n array A and key
- 3): Set i = 0
- 4): compare A[i] with key
- 5): if A[i] == key, display the position and go to step 8
- 6): increment i by 1 and repeat steps 4-5 until i < n
- 7): display "element not found"
- 8): stop.

code :

```
#include <iostream>
using namespace std;

int main() {
    int n, arr[100], key;

    cout << "Enter size of array: ";
    cin >> n;
```



```
cout << "Enter array elements: ";
for(int i = 0; i < n; i++)
cin >> arr[i];

cout << "Enter element to search: ";
cin >> key;

for(int i = 0; i < n; i++)
{
    if(arr[i] == key)
    {
        cout << "Element found at position " << i + 1;
        return 0;
    }
}

cout << "Element not found";

return 0;
}
```

OUTPUT :

```
Enter size of array: 5
Enter array elements: 4
23
45
63
21
Enter element to search: 45
Element found at position 3

...Program finished with exit code 0
Press ENTER to exit console.
```

time complexity (Step Count / Frequency Count Method)
best case :



linear search
(Best case)
element is found at
the first position
for (i=0; i<n; i++) {
 if (arr[i] == key) {
 return i;
 }
}
T(n) = 1
= O(1)

average case : Element is somewhere in the middle

linear search :-
for (i=0; i<n; i++) {
 if (arr[i] == key) {
 return i;
 }
}
return -1;
T(n) = O(n).

worst case : Element is at the last position / not found



Linear search :-
for (i=0; i<n; i++) - h
{
if (arr[i] == key) - 1
{
return i; - 1
}
return -1; - 1
}
T(n) = O(n).